ChatGPT

LastWeekIn.Tech – Weekly Tech News Aggregator (Product Spec & Architecture)

Introduction

In today's fast-paced AI era, technology news is overwhelming. **LastWeekIn.Tech** is a proposed solution to cut through the noise by curating only the **7 most important tech stories** of the week, updated every Monday. This web app will emphasize quality over quantity – delivering a concise weekly digest (at least half of the stories will be AI-related) with brief AI-generated summaries and links to original sources. By focusing on just **7 top stories in 7 days**, LastWeekIn.Tech aims to inform readers of crucial developments without information overload.

Goals and Key Features:
- **Weekly Top-7 Digest:** Every Monday, the site updates with *exactly seven* of the most influential tech stories from the past week (with ~50% or more focusing on AI topics). Content remains static through the week.
- **High-Quality Sources:** Stories are selected from reputable, high signal-to-noise sources (e.g. Hacker News, top tech news sites, etc.), filtered by an automated pipeline that scores and ranks their significance.
- **AI Summarized Content:** Each story is accompanied by a concise summary generated by a local Large Language Model (LLM). Summaries highlight the main point of the news in a few sentences, and each item links to the original article for full details.
- **Simple Static Site:** The frontend is a clean, Material Design-inspired Vue.js app. It displays the weekly list with a modern, *mobile-friendly* UI and the **Titillium Web** font for a stylish look. There is no complex interactivity or login – just content.
- **Fully Automated Curation:** All curation, fetching, and summarization are handled by a backend batch pipeline (Python + SQLite + LLM). No manual editing or admin UI is required; the system runs on schedule, producing a JSON file of the week's content which is deployed to the site.

By adhering to these goals, LastWeekIn.Tech will provide a **"last week's best"** snapshot to help tech enthusiasts stay informed with minimal effort.

Requirements Overview

Functional Requirements

1. **Aggregating Top Stories:** The system must gather candidates from multiple sources (e.g. Hacker News, RSS feeds of tech publications, etc.) for the past 7 days.
2. **Story Selection Algorithm:** Implement a scoring/ranking mechanism to identify the top 7 stories of the week. The algorithm should ensure *at least 3-4 of the 7 stories are AI-related*, reflecting the dominance of AI in tech news.
3. **Content Summarization:** For each selected story, generate a short summary (e.g. 2-4 sentences) using an LLM. The summary should capture the essence of the story without personal bias or hallucination.

4. **Static Content Output:** The weekly results (titles, summaries, source info, and original links) are compiled into a structured JSON (or similar format) and published to the website. This JSON is updated once a week (on Mondays) and remains constant until the next update.
5. **Frontend Display:** The Vue.js frontend loads the weekly content and presents it in a user-friendly list. Each story is shown with its title (clickable link to original), summary, source name, and possibly a category label (e.g. "AI"). The interface should be intuitive and visually appealing, using Material Design principles (e.g. card layout, consistent padding/spacing) and the Titillium Web font for a modern touch.

Non-Functional Requirements

- **Simplicity & Maintainability:** Follow KISS (Keep It Simple, Stupid) – avoid over-engineering. The architecture should be as simple as possible while meeting requirements. Use DRY (Don't Repeat Yourself) principles in code and a clear separation of concerns (e.g. data gathering vs. presentation) for maintainability.
- **Domain-Driven Design:** Model the core concepts (articles, stories, sources, etc.) in a clear domain layer, decoupling business logic from implementation details. This aligns with DDD and DIP (Dependency Inversion Principle) – for example, define interfaces for fetching sources and summarizing content so implementations can be swapped or extended without changes to high-level logic.
- **Performance:** The backend pipeline may process hundreds or thousands of articles to curate the top 7, which is acceptable since it runs offline on powerful local hardware. The pipeline can run for hours if needed, but it must complete before Monday publication. The web frontend, by contrast, should be lightweight and fast (all content is static or loaded from a static JSON, enabling quick page loads via CDN/Vercel).
- **Cost Efficiency:** Leverage local resources to minimize recurring costs. Use **local LLM models** for summarization and analysis to avoid API fees (e.g. run a Llama 2 variant or other open model on the user's hardware). The use of external APIs (like news feeds) should stick to free tiers or minimal use. Deployment on Vercel should be within free tier limits (static hosting).
- **Scalability & Extensibility:** While initial scope is a single static site updated weekly, the design should allow future extensions, such as: adding more sources, changing the number of stories, introducing categories or archives, or even moving to a dynamic backend if needed. By adhering to DIP and modular design, components like "scoring algorithm" or "summarizer" can be improved or replaced independently.
- **No Runtime Admin Interface:** All curation is automated. There is no need for users or admins to input anything on the site. New content is deployed via the pipeline pushing updates to the repository. (In the future, an admin review step could be added by editing the JSON before publish, but this is out-of-scope initially.)

Tech Stack Decisions

- **Backend Pipeline: Python** will be used for the data processing pipeline due to its rich ecosystem for web scraping, data analysis, and ML/LLM integration. A **SQLite** database will serve as a lightweight local store for articles and intermediate data (ensuring simplicity and zero setup). Key Python libraries likely include: `feedparser` or requests for fetching feeds/APIs, `newspaper3k` or similar for article text extraction, `sqlite3` for the DB, and HuggingFace Transformers or `llama.cpp` for running LLM models locally.

- **LLM and NLP:** Use local LLM models for summarization (and possibly classification), to avoid API costs. For example, a model like **Facebook BART-large-CNN** (which is optimized for summarization) can be run via HuggingFace Transformers to produce concise abstracts [1]. Alternatively, a fine-tuned Llama 2 or GPT-J model could be used for summarization instructions. The pipeline can also leverage simpler NLP for tasks like keyword extraction or deduplication (e.g. using `nltk` or `scikit-learn` for text similarity).
- **Frontend: Vue.js** (possibly with Vuetify or another Material Design component library for rapid UI development) will be used to build the user interface. The app will be a static single-page application (SPA) or a statically generated site (no server-side code) deployed on **Vercel**. The design will follow Material Design guidelines (e.g. using cards, responsive grid) and include the **Titillium Web** font from Google Fonts for a clean, modern aesthetic.
- **Deployment: Vercel** will host the static site. Weekly data (the JSON content) will be committed to the Git repository powering the site; Vercel's continuous deployment will publish the changes automatically. This means no live database or server is needed online – the site simply fetches the static JSON. In the future, if traffic grows or dynamic features are needed, a CDN or edge functions can be introduced to cache and serve content efficiently, but initially the static hosting on Vercel is sufficient.
- **Version Control & CI:** The code (frontend and pipeline) will reside in a repository (e.g. GitHub). The content JSON for each week can also be stored in the repo (e.g. in a `/data` folder). A simple CI/CD workflow can trigger on push to run tests or even run the pipeline (though more likely the pipeline runs manually on local and then pushes data). Optionally, Git LFS or a separate branch could store historical JSON if archiving old weeks.

Domain Model Design

To keep the design clear, we identify a few core domain entities and their relationships. This helps apply DDD principles by giving each concept a clear structure and responsibility:

- **Source** – Represents a news source or feed of stories. For example: a Hacker News feed, a specific RSS feed (TechCrunch, The Verge, etc.), or an aggregator API. Each Source may have attributes: `name`, `type` (e.g. "HackerNews", "RSS", "API"), and configuration details (URL or API endpoint, parser rules if any).
- **Article** – Represents a raw news article fetched from a Source. Attributes include: `id` (internal), `source` (link to Source), `title`, `url` (link to original content), `published_date`, and full `content` (the raw text or HTML of the article). It may also have metadata like `author` or `source_rank` (if the source provides a score, e.g. HN points). Articles are the inputs to our pipeline (prior to filtering and summarization).
- **Story** – Represents a *deduplicated news story* or event. In many cases, a Story maps 1:1 with an Article (when that story is only covered by one source). But if multiple articles from different sources refer to the same underlying event, they would be merged into one Story. A Story might contain: a `title` (perhaps taken from the most descriptive article or a combination), a set of one or more `articles` (references to Article objects that were deemed duplicates/related), a computed `score` (the outcome of our ranking algorithm for how important this story is), a `category` or tags (e.g. "AI" vs "General Tech"), and a `summary`. In essence, *Story* is the entity we ultimately want to output (7 of these each week).
- **Summary** (value object) – The text summary for a story. This is typically just a field on the Story (e.g. `story.summary_text`), generated by the LLM from one or more article contents. We treat it as a

distinct concept to emphasize it's produced data (not directly scraped). The summary is an **abstractive** synopsis of the story's content.

- **Edition** – Represents a weekly edition of LastWeekIn.Tech. An Edition might have a date (e.g. Monday's date or week number) and contain a list of exactly 7 Stories. In the simplest implementation, we might not explicitly model Edition in the code (we can just handle one edition at a time). However, treating it as an entity could help if we plan to archive multiple weeks or reference previous editions. For now, the Edition concept mainly maps to the JSON output structure (where we might have `{ week: "2025-10-27", stories: [ ...7 stories... ] }`).

These domain entities guide the structure of our code. For example, we might have Python classes or dataclasses for `Article` and `Story`. The logic of selecting top stories would operate on collections of Article objects and produce Story objects. By keeping domain objects simple (just data + minimal behavior), and handling selection logic in service classes/functions, we maintain a clean separation that aligns with DDD principles.

High-Level System Architecture

We can split the system into two major parts:

1. Offline Data Pipeline (Content Curation Backend) – a Python-based pipeline that runs on a schedule (or manually) on the developer's machine. It fetches and processes news data, applies ranking logic, uses LLM for summaries, and outputs the final JSON. This pipeline uses a local SQLite database for intermediate storage and can run for an extended period (hours if needed) since it's offline. The pipeline's stages ensure data quality and apply business rules (e.g. ensuring enough AI content).

2. Frontend Web Application (Static Site) – a Vue.js application deployed on Vercel that presents the content. It either fetches the JSON produced by the pipeline or has it pre-bundled. The frontend is purely for presentation; it does not do heavy processing – it simply loads the list of 7 stories and displays them in a user-friendly way. Material Design components and responsive design make it visually appealing on both desktop and mobile.

Below is an overview of the **data flow** and architecture interactions:

- **Data Pipeline Steps:**
- **Fetch Sources:** On (or before) Monday, the pipeline gathers raw articles from various sources covering the last week. For example, it might call the Hacker News API for top stories, request RSS feeds from tech news sites, or query a news API for top tech headlines [2] [3]. All fetched articles (title, link, etc.) are stored in a local SQLite DB or in-memory structures for processing.
- **Content Extraction:** For each fetched article, the pipeline retrieves the full content text. RSS feeds or APIs often provide only a snippet or truncated content [4], so the pipeline will perform an HTTP GET on the article's URL and use an HTML parsing library (like `newspaper3k` or `BeautifulSoup`) to extract the main text content. This ensures the summarizer has the complete article text to work with, not just a summary or title [4].
- **Normalization & Cleaning:** The raw text is cleaned – remove any HTML tags, scripts, or noisy content (navigation menus, etc.) using the parser's output or custom rules. Very short or malformed articles might be dropped here (e.g. if an article has only one sentence, it's likely not useful). Non-

English articles (if any) would be filtered out since our audience is assumed English. We also ensure each article has a valid date within the last week.

- **Deduplication / Story Clustering:** The pipeline identifies if multiple articles are about the same news story. For example, "OpenAI releases GPT-5" might be covered by TechCrunch and also discussed on Hacker News; we should treat that as one story to avoid redundancy. Deduping can be done by comparing titles and content. A simple approach: compute a hash or similarity score for titles [3] – if two titles share a lot of uncommon words or have high similarity (or if one title is contained in another), consider them the same story. More robust approach: use embeddings (e.g. sentence transformers) to cluster articles by content similarity, or even use an LLM to compare if two articles describe the same event. Initially, a straightforward fuzzy match or hashing of titles and URLs will be used (e.g. ignore minor punctuation or "| TechCrunch" suffixes in titles). The SQLite DB can help by storing seen URLs or content hashes to avoid duplicates. (The n8n community example uses a hash of the URL to skip duplicates in Notion [3].) Each cluster of duplicates will form one *Story* in the next stage.
- **Scoring & Ranking:** Each Story (or single Article if no duplicates) is then evaluated for importance. This is the heart of the algorithm. We'll incorporate multiple signals:
    - **Hacker News Points:** If the story came from Hacker News, we have a score (points/upvotes). A high point count (especially within the past week) strongly indicates significance in the tech community. We can normalize HN scores to a certain range for comparison.
    - **Multi-Source Presence:** If the story was found on multiple sources (e.g. several news sites wrote about it), that boosts its importance. We might increment a story's score for each distinct source that covered it.
    - **Source Authority:** Optionally, weight certain sources higher. For example, a story from an official source (like a major publication's top headlines) might inherently carry weight. However, since we're already curating sources for quality, we might treat all sources more or less equally and let frequency of mention determine importance.
    - **Recency/Relevance:** All stories are within a week, but something breaking later in the week might have fewer total mentions yet still be very important. The algorithm may account for story age (e.g. slightly favor things that broke very recently if they're trending upward). However, because we only update weekly, we primarily consider the whole week.
    - **AI Priority Bias:** To satisfy the requirement that ~half the stories are AI-related, we will incorporate a bias for AI content. Implementation: we tag each story as "AI-related" or not (using either a simple keyword check or a classification model). For instance, if the title or content mentions "AI", "artificial intelligence", "machine learning", "GPT", "OpenAI", etc., we mark it as AI. We can also leverage an LLM in zero-shot mode to categorize a summary into "AI" vs "General Tech" for higher accuracy [5]. During scoring, we can give a small bonus to AI-tagged stories to increase the likelihood that some of them rise into the top 7. (Additionally, after initial scoring, we will enforce at least 3 AI stories by adjusting the list if needed – see selection step.)
    - **Engagement Metrics:** If available, use other metrics such as number of comments (HN or Reddit comments), social media share counts, or article view counts. These are not always accessible, but Hacker News comments or Reddit upvotes could be fetched for those sources and considered. For MVP, we might rely mostly on HN points and multi-source mentions as proxies for engagement.
    - The output of this stage is a **score** for each story that allows us to rank them. For example, one simple heuristic:

```
story_score = log2(HN_points + 1) + (number_of_sources * 2) +
```

`(AI_bonus)`. This is a rough idea: a story gets points for HN popularity, extra weight if multiple sources covered it, and a small bonus if AI-related. We will refine this formula with testing (possibly adjust weights).

- **Top-7 Selection:** We sort the stories by their computed score (or rank). Then pick the top 7. We then enforce business rules on this set: if fewer than 3 of the 7 are AI-related, we will consider swapping in the highest-scoring AI stories from rank 8+ to replace the lowest-ranked non-AI stories, until we have at least half AI stories. (We must ensure the replacements are still reasonably high scoring; we won't include irrelevant AI content just to hit a quota.) The final chosen 7 should each be significant, distinct stories that together give a well-rounded snapshot of the week. The stories can be ordered by significance (highest score first) or perhaps in chronological order of the week. Likely, we'll list by significance so that the #1 story is truly the biggest news.
- **Summarization:** For each of the 7 final stories, generate a concise summary using the LLM. We will feed the **full article content** (or a trimmed version if too long) to the model with a prompt like: *"Summarize the following news article in 2-3 sentences, focusing on the key point and outcome."* The LLM will then produce an abstractive summary. Using a local model (e.g. BART or Llama 2), we ensure no external API costs. We might have to run each summarization sequentially due to resource limits, but summarizing seven articles is manageable. Summaries should be factual and capture the essence (e.g. "OpenAI released the GPT-5 model, which offers 10× the parameters of its predecessor, aiming to improve reasoning. The release has significant implications for AI applications in healthcare and finance."). We will instruct the model not to include unrelated commentary and to keep it short [6]. If the model we use has context length limitations, we may summarize a subset of the content: often the first few paragraphs of a news article cover the main points. Alternatively, we could run an extractive summary first (like take the article's own introduction or use TextRank to get important sentences) and feed that to the LLM for polishing. For the MVP, a direct abstractive summary with a decent model is the plan [1].
- **Output Preparation:** Each story is now finalized with: title (we might use the original title or a slightly modified one for clarity if needed), source (e.g. "(TechCrunch)" or "via Hacker News"), the URL to the original story, and the AI-generated summary. We compile these into a JSON structure, for example:

```
{
  "week": "2025-11-03",
  "stories": [
    {
      "rank": 1,
      "title": "OpenAI Releases GPT-5, Ushering in Next Generation AI",
      "source": "TechCrunch",
      "url": "https://techcrunch.com/2025/10/30/openai-releases-gpt5...",
      "category": "AI",
      "summary": "OpenAI has unveiled GPT-5, a new AI model with ten times
the parameters of GPT-4 and improved reasoning abilities. The release is
expected to accelerate AI adoption in industries like healthcare and
finance."
    },
    { ... story 2 ... },
    ... { story 7 }
```

```
  ]
}
```

This JSON is written to a file (e.g. `latest.json` or `2025-11-03.json`). We also update an index file or a constant in the frontend so that the site knows which week/current JSON to display. (Alternatively, `latest.json` could be overwritten each week, and the frontend always fetches that fixed filename.) Historical JSON files could be kept for archive purposes, but initially the focus is on the latest.

- **Deployment:** Finally, the new JSON (and any updated code if needed) is pushed to the repository. This triggers Vercel to deploy the updated site. Since the site is static, deployment might simply upload the new JSON and any prebuilt assets. The pipeline run can be automated via a cron on the developer's machine every Monday early morning, or it can be initiated manually with a script when ready. With everything automated, by Monday 8am for example, LastWeekIn.Tech will have the new top-7 stories visible to everyone.

The above sequence covers the *backend workflow* in detail [2]. It ensures data quality by cleaning and deduplicating, and leverages both algorithmic ranking and LLM capabilities for summarization (and possibly classification). Crucially, this pipeline is *idempotent* for each week's data – you can re-run it and get the same result (assuming sources don't change), and it doesn't maintain complex state between weeks (aside from perhaps remembering past story URLs to avoid repeats if desired). This statelessness (per week) keeps things simple.

Illustration of Data Pipeline (Simplified):
Sources (HackerNews API, RSS feeds) → Fetch articles → SQLite (raw articles) → Clean & Deduplicate → Score & rank stories → Select top 7 (adjust AI ratio) → LLM summarization → Output JSON → Deploy to site.

This modular pipeline adheres to DRY – e.g., common fetching/parsing code is reused for multiple sources – and uses DIP by isolating source-specific logic. For instance, adding a new source feed later would involve writing a new fetcher that implements a common interface (say `ArticleFetcher`) and plugging it in, without altering the core ranking algorithm.

Frontend Architecture and Design

The frontend is a static **Vue.js** application that presents the weekly stories in a clean, scrollable view. Key aspects of the frontend design and architecture include:

- **Static SPA:** We will create a single-page application with Vue (using either Vue CLI or Vite). It will likely be deployed as a static bundle (HTML/JS/CSS) on Vercel. On load, the app will fetch the JSON data for the current week (e.g. via an HTTP GET to `latest.json` or embed it during build). Because the content changes only weekly, we could even embed the JSON at build-time – but fetching it allows a decoupling where the data file can be updated independently.
- **State Management:** This app is simple enough that we don't need Vuex or global state management. The list of stories can be stored in a component's local state once fetched. No complex user interaction or state transitions are needed beyond maybe a loading indicator while fetching data.
- **Components:** We will create a few reusable Vue components for clarity:

- `<StoryList>` – a component that given an array of story objects, renders the list. It may simply map each story to a `<StoryCard>` sub-component.
- `<StoryCard>` – a component to display a single story in a Material Design "card" style. It will show the title (as a link), the summary text, and possibly a subtitle with the source name and category badge (e.g. a small "AI" chip if the story is AI-related). It could also show the published date if relevant (though all stories are within one week, date might not be critical, but could be included for context). The card will use consistent styling (e.g. a slight elevation, padding, etc., typical of Material cards).
- (Optional) `<Header>` – a simple header bar with the site title "LastWeekIn.Tech" and maybe the edition date ("Week of Nov 3, 2025").
- (Optional) `<Footer>` – could contain a note like "Updated every Monday • Sources: AI-curated from across the web" or any attribution.

- **Material Design & Styling:** We will apply Material Design guidelines for typography, spacing, and color. Possibly use **Vuetify** (a popular Vue Material Design component framework) to expedite this – Vuetify provides ready-made components like `<v-card>` and layout grid, which would ensure a polished look out-of-the-box. If using Vuetify, we can configure the theme to use Titillium Web for headings/body text. Alternatively, a lighter approach is to use Google's Material design web components or just CSS utilities. Regardless, the design will have a **responsive layout** (cards stack vertically on mobile, maybe in a grid on larger screens). We'll use plenty of white space and a neutral color scheme (maybe a light mode with a white background and a primary color for accent, plus a dark mode toggle if time permits). Titillium Web font will be imported via CSS from Google Fonts.

- **Interaction:** The site is mostly read-only. When the app loads, it will fetch the JSON (using Axios or fetch API), and then populate the story list. Each story's link opens the original source in a new tab when clicked. We might add a hover effect or an icon for external link to clarify that. If we include archives in the future, there could be a dropdown or list of past dates to switch the view, but initially we focus on the current week only.
- **Performance:** Since content is limited (just 7 items of text), the app will be very lightweight. We should ensure the bundle size stays small (maybe use Vue's production build and only the necessary polyfills). Using a CDN (which Vercel effectively does) means the site will load quickly worldwide. We also ensure images or heavy assets are minimal – possibly each story might have a thumbnail image if available, but that complicates things (we didn't gather images in pipeline, and material design can work with a text-only card too). To keep it simple, we will not include images in the first version. This avoids having to fetch images or deal with their layout. The focus remains on text content.
- **Error Handling:** If the JSON fails to load (network issue or if no update for a week), the app should handle it gracefully – e.g. show an error message or show last known content if we embedded it. Because we plan the JSON to be updated reliably, this is a minor concern. We can implement a basic "try again" or display an apology message if needed.
- **SEO & Metadata:** Even though it's a SPA, we might use Vite or Nuxt for pre-rendering to ensure the content is crawlable (important if we want the site to be discoverable on search engines). Another approach: deploy the site as static HTML that already contains the week's 7 stories (instead of loading via JS). This could be done by generating the HTML at build time using the JSON. Since we have a weekly update cycle, we could incorporate a build step where the pipeline or CI injects the content into an HTML template. However, this adds complexity to automation. Given SEO is not a

primary requirement in the prompt, we can stick to the SPA approach for now and add prerendering later if needed.

- **Testing:** We will test the frontend on various devices and screen sizes to ensure the design looks good. Also test that the JSON parsing is correct and all fields are displayed. Because content updates weekly, the site should be resilient to slight variations (e.g. if one summary is a bit longer, the card should expand properly; if a title is long, it might wrap, etc.). Using Material components or good CSS will handle these.

Deployment on Vercel: The static site will be connected to the GitHub repo. Each push triggers a deploy. When the pipeline adds a new JSON and perhaps updates a constant (like `currentWeek = '2025-11-03'`), the commit triggers a new build, and Vercel serves the updated app. Over time, if we keep old JSON files, we could allow routes like `/2025-11-03` to access older editions. Vercel (with a frontend router in Vue) can handle that by serving the same index.html for those routes and the app determining which JSON to load based on route param. But again, initial scope can just show the latest edition.

In summary, the frontend is straightforward: it **consumes the prepared data and focuses on presentation**. The heavy lifting (aggregation, filtering, summarizing) is all done by the backend pipeline. This separation ensures that the web UI remains snappy and uncomplicated, since all it does is render static content updated periodically.

Data Pipeline Detailed Design

Diving deeper into the backend pipeline design, we outline the implementation plan for each stage, including specific technologies, data models, and how to keep the design modular:

1. Source Selection & Fetching

We will incorporate multiple high-quality sources. Initially identified sources include:
- **Hacker News (HN):** Known for surfacing important tech topics with high signal. We can use the official HN API or the Algolia search API to get top posts of the week. *Implementation:* Use the Algolia API to query stories from last 7 days sorted by points (there's a parameter for date range and sorting by popularity). Alternatively, use Firebase API to get top 500 stories, then filter those by timestamp (past week) and then sort by score. HN data gives us title, URL, score, comment count, etc. We'll retrieve maybe the top ~50 HN stories of the week as candidates.
- **RSS Feeds of Tech News Sites:** We'll pick a handful of respected tech news sites such as **TechCrunch**, **The Verge**, **Wired**, **Ars Technica**, **MIT Tech Review**, etc. Each typically offers an RSS feed for their latest articles. We can use Python's `feedparser` to fetch recent entries. Since we only want the past week, and these feeds are chronological, we can fetch daily or fetch once and filter by date. We might gather ~10-20 articles per source per week, resulting in ~100+ candidates. Each feed entry gives us title, link, publish datetime, and maybe a summary or excerpt. (We will later fetch the full content from the link.)
- **AI-Focused Sources:** To ensure we catch AI-related news, we might include one or two specialized sources. For example, the **r/MachineLearning** subreddit top posts of the week (if accessible via Reddit API or Pushshift) could highlight AI research news. Or an AI news blog like **Import AI** (if they have RSS). However, many major AI announcements also appear on general tech sites and HN, so this may be optional. We could use a keyword approach: e.g. query NewsAPI for "AI" in technology category. For now, including HN plus mainstream tech feeds should suffice, since HN is very AI-attentive lately.

- **Optional – News API:** As an alternative to manually curating site feeds, we can use an aggregator API such as **NewsAPI.org** or similar. For example, NewsAPI has an endpoint for top headlines in technology. It could return a list of articles from various outlets for the past day; by running it daily or using their everything search with a date range, we can get a broad set. The Medium article example used NewsAPI and then scraped full text [7]. We have to mind rate limits and the fact that content from NewsAPI may be truncated, requiring a second step to fetch the article [4]. Since our pipeline is offline and can take its time, doing the additional fetch is fine. If using NewsAPI, we could retrieve, say, 100 articles from the week and add to our pool. This might overlap with the RSS sources, but that's okay (deduplication will merge them).
- **Dedicate Quality Control:** We will avoid low-quality sources or pure PR news. The selection of feeds/APIs can be tuned. For instance, HackerNews and a curated set of top tech publications already filters out a lot of junk. If we see noise slipping in (like trivial gadget releases or too many startup funding announcements), we might refine our source list or add filtering rules (e.g. ignore news that are essentially press releases or minor updates).

Fetching Implementation:
We'll implement a set of classes or functions for fetching sources, for example:

```
class SourceFetcher(Protocol):
    def fetch_articles(self) -> List[Article]: ...
```

We then implement concrete fetchers: `HackerNewsFetcher`, `RSSFetcher`, `NewsAPIFetcher`, etc., each returning a list of Article objects with at least title, url, date, source info. These fetchers encapsulate the details of connecting to APIs or parsing feeds. By using an interface (protocol or abstract base), we adhere to DIP – the pipeline logic will call `fetcher.fetch_articles()` without needing to know if it's RSS or HN etc. Adding a new source later is as simple as adding a new class and including it in the list of fetchers to run.

We'll also store results in **SQLite** for analysis and potential reuse. The SQLite schema might have a table `articles` with columns: `id` (PK), `source_name`, `title`, `url`, `published`, `content`, `hn_points`, `hn_comments`, `processed` (boolean). Initially when fetching, we fill in what we have (for RSS/NewsAPI, we might not have points; for HN we have points but not content). Later steps will fill in the `content` and mark processed or attach summaries. SQLite is helpful for querying (like finding duplicates or ordering by date) using SQL – but it's optional if in-memory structures suffice. Given potentially a few hundred articles, in-memory is fine; SQLite mainly ensures persistence if we run the pipeline incrementally.

2. Content Extraction

After initial fetch, we will have many articles with just meta-data. Now, for each unique article URL, we retrieve the full text content:

We will use the **newspaper3k** library (or similar) which is designed to extract the main body text from news article pages [4]. Newspaper3k can take a URL and attempt to download and parse the article, returning the text content, title, authors, etc. It works for many news sites by using heuristics (looking for `<p>` tags, common article tag ids, etc.). This saves us from writing custom scrapers for each site. We need to be mindful of rate limits and politeness (we should insert small delays between requests or limit concurrency so as not to hammer any one site's server, especially since we might fetch dozens of pages).

Alternatively, some RSS feeds include the full content or at least a substantial excerpt – if so, we could use that content directly for summarization. But to ensure quality, we'll generally fetch the actual page. The n8n workflow example shows using HTTP requests and CSS selectors for each site [8]; using newspaper3k is more general and in line with KISS/DRY (one library to handle all). If newspaper3k fails on a site or gets blocked, a backup approach is to use requests + BeautifulSoup and attempt to find article text (e.g. all `<p>` tags inside the main `<article>` tag). For Hacker News items, the URL is often an external link (which we treat like any other article). If an HN item is actually a text post (rare for top stories), we can retrieve its text via HN API as a special case.

We will store the extracted content in the `content` field of our Article (or a separate table if needed). After this step, each Article has the raw text needed for summarization. We also might compute a quick fingerprint of the content (like a hash of the text or just the first sentence) to help identify duplicates in the next step.

3. Deduplication and Clustering

Now with content and titles in hand, we perform deduplication to form *Story* groups. Deduplicating can be approached at two levels: - **Exact duplicates:** If the same article appears twice (e.g. maybe fetched from two different sources or the same article was referenced multiple times), identify by URL or title exact match. Using the URL hash (as in the n8n example) is an easy way to skip exact duplicates [3]. Our pipeline can enforce uniqueness by making the URL the primary key in the DB or by ignoring articles with a URL already seen. This addresses duplicates like a single story showing up in both an RSS feed and NewsAPI results.
- **Same story, different sources:** Harder, since titles and content will differ but refer to the same event. For example: "Google announces new AI chip" vs "Google's new AI chip unveiled to boost data centers" – different wording, same news. We will do a simple clustering: - **Title similarity:** We can normalize titles (lowercase, remove punctuation, remove stopwords) and check overlaps. If two titles share a rare keyword or phrase (like a product name), it's a clue. A straightforward method: for each title, create a set of keywords (excluding common words). Compute Jaccard similarity between title sets. If above a threshold (say > 0.5) then consider them related. Alternatively, use Python's `difflib.SequenceMatcher` to get a similarity ratio between strings.
- **Content summary similarity:** Another trick is to quickly generate a lightweight representation of content. For instance, take the first 100 words of each article (which often contain the core subject). Compute a TF-IDF or even a simple vector (bag-of-words) and compare cosine similarity. If high, cluster them. Given potentially not too many articles, even an O(n^2) comparison is fine (n maybe a few hundred).
- **Clustering algorithm:** We can use a union-find or simply mark duplicates as we find them. We'll iterate through articles by date or score and cluster related ones. Each cluster can be represented by a Story object containing all member Articles. We might pick one representative article (perhaps the one with the highest HN score or the most detailed content) to use for summarization. Alternatively, we could combine info (but to keep it simple, just pick one).
- **LLM approach (optional):** For higher accuracy, we could use an LLM to check if two articles are about the same thing: e.g. feed both titles or a summary of both to a prompt: *"Do these refer to the same news event? Title1: ..., Title2: ..."*. However, doing this for every pair is expensive and probably overkill. Our simpler methods above should catch obvious overlaps.

After this stage, we have a list of **Story** clusters. Each Story has one or more sources/articles. For each Story, we might choose to keep the title of the most prominent article (or even create a new title, but it's probably

fine to use an existing headline from one source – often the wording from a major site is clear and catchy). We'll also aggregate any metadata: e.g. if one story had an HN score, attach it to the story for scoring; if multiple sources, note the count.

4. Story Scoring Algorithm

As outlined in the overview, we will score each Story to determine its importance. Let's formalize a possible implementation:

Suppose for each Story we have: `num_sources`, `max_hn_points` (max points among member articles, or 0 if none from HN), `avg_hn_points`, `published_date` (could use the earliest or latest pub date among articles), `category` (AI or General). We might also have other flags like if it was widely shared on social (if we integrate that later).

We compute a base score. One approach in code:

```
score = 0.0
score += min(max_hn_points, 300) / 30.0   # scale HN points (cap at 300 to not
let one story dominate too much)
score += num_sources * 5                  # each additional source gives a flat
boost
if category == 'AI':
    score += 2                            # small bonus for AI stories
# Optionally adjust for recency: newer stories might get +1 if within last 2
days, etc.
```

This is just an example. The idea: an HN story with ~300 points (which is huge) would get about 10 points from that factor, whereas a story covered by e.g. 4 sources gets +20. We'd tune these weights so that typically the top story might have a score in the dozens. We then rank by this score.

We will try out the algorithm on some historical data (this is where having the pipeline easily rerunnable is handy). If it seems off (e.g. it picks something trivial as #1 because it was on 5 small sources), we adjust weights. We can also impose some manual tweaks: e.g. ensure that if something was #1 on Hacker News with 500 points, it *will* be in the top few regardless of how many sources mention it, because HN high rank is a strong signal of importance to the tech community.

We also incorporate the **AI bias** either directly as above (small bonus) or indirectly by final selection rule. Since we plan a final check anyway, the bonus just helps get those AI stories into contention.

5. Selecting Top 7 and Ensuring AI Balance

Once scores are computed, sort stories in descending order. We select the top 7. If the 7th and 8th are very close in score, it's not critical, but we stick to 7 for the concept of "top 7".

Now enforce the rule: count how many of these 7 are categorized as AI. If fewer than 3 (half of 7 rounded down) are AI, we will promote some AI stories from below. How to do this systematically:

- Suppose out of initial top 7, only 2 are AI. Then we need at least 2 more AI to reach 4 (half of 7 rounded up). We look at the list of stories ranked 8, 9, … and find the top AI-related stories among those. Pick as many as needed (in this case 2) and mark them for inclusion. Correspondingly, identify an equal number of stories from the current top 7 that are non-AI and lowest-ranked. We will remove those and replace with the AI ones. This way, we maintain the total at 7. Of course, we should only do this if the AI stories below are reasonably close in score; if an AI story is way down (meaning it wasn't very significant at all), forcing it in might reduce the overall quality. But given the prominence of AI, it's likely there will be some strong AI stories in the top 10 or 15 anyway. The bias introduced earlier also makes it more likely we had at least 3 to start with.
- If more than half are AI (say 5 out of 7), that's actually fine as per requirements (at least half *being AI means half or more*). So we don't need to limit AI, only ensure minimum. If one week all top stories are AI-related, we can actually run with it – the user's guideline was at least half, not exactly half. The ethos is to not miss key AI news, since that's a focus area.
- Documenting this rule clearly in code is important so future maintainers (or the AI assistant reading the spec) understand why we do this swap.

After this, we have our final 7 stories locked in. We can prepare the data for summarization: perhaps gather the full content of whichever representative article we will summarize for each story. (If a story cluster has multiple articles, we might choose the longest content or the one from the most trusted source for summarization).

6. Summarization with LLM

For each story, we will produce a summary. Implementation details: - **LLM Choice:** We prefer a local model to avoid API cost. Two good options: - **BART Large CNN** (which the Medium article used) – it's a 400MB model that can run on CPU (slowly) or GPU (faster). It's specialized for summarization and can handle fairly long texts (1024 token input). Using HuggingFace's Transformers library, we can load `facebook/bart-large-cnn` and call it in a pipeline for summarization [1]. This will yield a pretty decent summary since it's trained on news. The downside: it might produce a generic-sounding summary sometimes, but it tends to be factual.
- **Local GPT model (Llama 2 13B chat or similar)** – we could prompt a chat model to summarize. This might produce more natural language, but smaller local models might hallucinate or omit facts if the article is long. If we have a strong enough machine, a 13B or 30B model with 4-bit quantization could possibly summarize okay. But BART might still outperform on strict summarization quality for news, given it was made for that. A compromise: use the open-source **Pegasus** model or **T5** model fine-tuned on CNN/DailyMail dataset (also for summarization). These are available and not too huge.
- **Process:** We will iterate through each story: - Prepare the text: ideally the full article text. If the article is extremely long (some investigative pieces can be thousands of words), we might truncate to a limit (maybe 1000-1500 words) focusing on the beginning and any conclusion, since the core info is usually at the top.
- If using a summarization pipeline (like `summarizer = pipeline("summarization", model="facebook/bart-large-cnn")`), simply call `summary = summarizer(text)` which returns a condensed paragraph. We may then post-process that summary: e.g. ensure it's no more than ~3 sentences, and that it mentions the key entities (company or product names, etc.) so the reader knows what it's about. If the summary is too long or misses something obvious that was in the title, we might prepend the title or a key fact. But ideally, the model output is used as-is except maybe trimming.

- If using a chat model, we'll craft a prompt template like:

```
Summarize the following news article in 2-3 sentences, focusing on the main
point and any important outcomes. Do not add information not in the article.
Article: \"\"\"
[article text here]
\"\"\"
Summary:
```

And then get the completion. We'd likely run this via a library like `llama_cpp` for a local model or via Huggingface's `generate`.
- **Local Execution Performance:** Summarizing 7 articles with BART or a 7B-13B model is not too bad. If each summary takes e.g. 30 seconds on CPU, the whole summarization stage might be ~3-5 minutes, which is fine. If GPU is available, even faster. Since pipeline can run for hours, this is negligible in context. We just have to ensure memory is sufficient (some models may use a lot of RAM – BART and T5 are manageable though).
- **Quality Check:** It's wise to log or save the raw model output and maybe do a quick sanity check. At minimum, ensure the summary is not empty and not ridiculously off-topic. We can automatically check if certain key terms from the title or content appear in the summary (they usually should – e.g. if the story is about "Google X", the word "Google" should likely appear in summary). If a summary fails this check, we could consider re-summarizing or falling back to an extractive approach (like just take first sentence of the article which often is a decent summary). However, presumably the LLM will do fine for most news pieces.

We will attach each summary text to the respective Story object.

7. Data Model and Interfaces (Summary)

Bringing it together, here's a brief look at the Python classes and their key methods that implement the above:

- `Article` (dataclass): fields like `id, title, url, content, source_name, published_at, hn_points, hn_comments, is_ai` (the last one might be derived later by checking content for AI keywords).
- `Story` (dataclass): fields like `title, articles (List[Article]), score, category (AI/General), summary`. Possibly a method `compute_title()` that picks the best title from articles (or just use the first).
- `SourceFetcher` (abstract base or protocol): method `fetch_articles() -> List[Article]`. Concrete implementations:
- `HackerNewsFetcher`: uses HN API to get top stories of last week, returns Article list with HN points filled in (but content empty initially, except maybe the HN text if any).
- `RSSFetcher`: initialized with an RSS URL (and source name), uses feedparser to get entries, returns Article list.
- `NewsAPIFetcher`: uses NewsAPI client to get articles for the week, returns Article list.
- `ContentExtractor`: class with static method `extract(article: Article) -> str` that uses newspaper3k or requests/BS4 to get the text. Could also integrate with Article (like an `article.fetch_content()` method).

- `StoryClusterer`: a service class with method `cluster_articles(articles: List[Article]) -> List[Story]`. Implements dedup logic: returns a list of Story objects (initially without summary or score).
- `StoryScorer`: class with method `score_story(story: Story) -> float` and maybe `score_stories(stories: List[Story]) -> None` that adds a score to each story (as story.score). This uses the algorithm combining HN points, source count, etc. as described.
- `Summarizer`: class with method `summarize_text(text: str) -> str`. Internally perhaps uses a loaded model (we could initialize the model once outside to avoid re-loading each time). We might implement this as a lightweight wrapper around the Transformers pipeline or any model interface we choose. Alternatively, use Huggingface's pipeline directly in code. A more advanced design: make `Summarizer` an interface too, so we can plug different summarization strategies (e.g. `OpenAISummarizer` vs `LocalLLMSummarizer`). For now, we can just implement one using local model.
- `PipelineOrchestrator`: This could be the main function or class that ties it all: it calls the fetchers, collates all articles, calls clustering, scoring, selects top 7, calls summarizer for each, then outputs JSON. If using a class, it might have methods for each stage, or just one big `run_pipeline()`. Since we want to keep it simple and script-like, a linear script in `if __name__ == "__main__":` could suffice, but encapsulating steps in functions helps testing. For example:

  ```
  all_articles = []
  for fetcher in fetchers:
      all_articles.extend(fetcher.fetch_articles())
  save_articles_to_db(all_articles)
  for article in all_articles:
      article.content = ContentExtractor.extract(article)
  clusters = StoryClusterer.cluster_articles(all_articles)
  for story in clusters:
      story.category = categorize_story(story)  # e.g., determine AI or not
      story.score = StoryScorer.score_story(story)
  clusters.sort(key=lambda s: s.score, reverse=True)
  top_stories = select_top_n(clusters, n=7, ensure_ai_half=True)
  for story in top_stories:
      story.summary = Summarizer.summarize_text(story.get_main_text())
  output_json(top_stories, week_date)
  ```

  Each of those steps uses the helper classes. This pseudo-code shows a clear flow and keeps each part relatively independent (which is good for maintainability and clarity).

The above design respects **DRY** by not duplicating logic – e.g., all RSS feeds can reuse one `RSSFetcher` class by just giving different URLs (could even make it data-driven: a list of feed URLs in a config file). **DIP** is applied by making fetchers and summarizer pluggable. For instance, if later we want to replace the local summarizer with an external API call (perhaps for better quality), we could implement a new `Summarizer` without changing the orchestrator logic. Similarly, if we find a better way to rank stories (say using a trained ML model), we could implement that in `StoryScorer` and not affect other parts.

8. Data Persistence and Pipeline Operation

Using SQLite means we can accumulate data gradually. One strategy: run the pipeline incrementally each day (fetch daily and insert into DB, mark those days done) then final scoring at week's end. But given the complexity, it might be simpler to run it all at once weekly. The SQLite is mainly for convenience of queries and maybe for caching content fetches (so if we rerun on same data it doesn't have to re-download everything). It's also nice for inspecting what was fetched (for debugging, we can open the DB and see what articles were considered).

However, an MVP might not strictly require writing to DB – we could hold everything in memory lists and just output JSON. The reason to keep DB could be to build an archive or to ensure idempotence if the script crashes mid-way (you could resume from partially filled data). In line with KISS, we might start without a persistent DB (just an in-memory structure), and then add SQLite if we need persistence. Since the user specifically mentioned SQLite, they likely expect it used, so we'll incorporate it at least as temporary storage.

Finally, after outputting JSON, the pipeline will **commit the JSON file to Git** (this could be automated using GitPython or simply by calling a shell command from Python). If we integrate with CI, an alternative approach is the pipeline pushes data to a cloud storage and the frontend fetches from there, but that's unnecessary here due to using repo/CI.

Seven Alternative Approaches (Architecture Options)

In designing this system, we considered multiple approaches to both the curation algorithm and the system architecture. Here are **7 potential architectures/strategies**, with their pros, cons, and an assessment of viability:

1. **Approach 1: Hacker News-Only Curation**
   Description: Use Hacker News as the sole input for finding top stories. Each week, simply take the top N posts on HN from the past 7 days (by score). Summarize those and publish. Possibly filter for AI content by picking at least half from the HN list that are AI-related.
   Pros: Very simple pipeline – HN provides a ready-made ranking by score, essentially crowdsourced importance. Only one source to fetch (reduces complexity), and HN's high signal-to-noise means many important stories (especially in tech and AI) will appear there. No need for complex scoring algorithms; just trust HN votes.
   Cons: **Coverage bias** – HN skews toward the interests of its community (lots of programming, startup discussions, some science); it might miss big mainstream tech news that HN users don't heavily discuss. Also, some HN top posts are not "news" (could be blog posts, tutorials, etc., which might not fit the "critical influential news" goal). Relying on one source is risky – if HN has an off week or certain biases, our output might not be well-rounded. Ensuring at least half AI might be tricky if HN's top doesn't have enough AI stories (we could force it, but then we're deviating from the HN ranking anyway).
   Probability of Success: Moderate. This approach will reliably produce content, but the quality and representativeness of the 7 stories might not meet the "top of the top across the tech world" ideal. It's the simplest but not the most comprehensive. It could serve as a baseline or fallback approach.

2. **Approach 2: Multi-Source RSS Aggregation**
   Description: Curate a fixed list of top tech news RSS feeds (e.g., TechCrunch, The Verge, Wired, Ars

Technica, Hacker News, perhaps a couple of AI blogs). Fetch all articles from these feeds for the week, then deduplicate and rank by a custom algorithm (likely based on how many sites covered the same story). Summarize the top 7.
Pros: Covers a breadth of perspectives – mainstream tech journalism plus community (HN). Likely to catch all major news since major outlets will report big stories. The algorithm can emphasize stories covered by multiple outlets, implicitly filtering importance (if 5 major tech sites all wrote about X, it's probably big). It's relatively straightforward to implement using feed parsing and doesn't depend on third-party APIs (RSS is open).
Cons: Need to carefully handle duplicates and variances in reporting. Also, we must maintain the list of feeds (if a site changes its feed URL or goes down, we update it). Some important sources might not have easily parseable feeds (or might have partial info requiring scraping). Ranking algorithm might still need fine-tuning (e.g., one site might report a niche AI research as a big article, but if others don't, it might be missed unless on HN). There's some complexity in merging different feeds and possibly weight them (should an article on a more authoritative site count more?).
Probability of Success: High. This approach likely yields a strong set of stories, as it mimics what human-curated digests (like TechMeme or newsletters) do – aggregating multiple sources. The technical complexity is moderate (RSS parsing and deduping logic), which is manageable. This is a promising approach and close to what we propose as the solution (with perhaps adding HN as one of the feeds).

3. **Approach 3: News API / Aggregator Service**
   Description: Use a third-party news aggregator API (such as NewsAPI.org, Google News API, or GNews) to fetch the top news of the week in technology (and possibly a separate query for AI). Essentially offload the gathering and initial ranking to an external service. Then use LLM to summarize the results.
   Pros: Very convenient – one API call could return a list of "top" articles, already drawing from multiple sources. Implementation is quick (just parse JSON from the API). It can cover many outlets including ones we might not think of. Also, some APIs allow sorting by popularity or relevance which helps get top stories without heavy custom logic.
   Cons: **Dependency on external service:** NewsAPI, for example, requires an API key and has rate limits and possibly costs if volume is high. Relying on it means if the service changes or the free tier isn't enough, the system breaks or costs money. Also, aggregator APIs might not exactly yield the "top 7" we want – they might return a lot of trivial news unless we fine-tune the query. We'd still need to filter and dedupe. Content may be truncated [4], forcing us to scrape anyway (as seen in similar projects). Additionally, the definition of "top" might just be based on general popularity, which might lean towards more mainstream news and ignore niche but important tech dev stories that HN would catch.
   Probability of Success: Moderate to High. It will fetch news successfully, but achieving the right blend and ensuring the AI ratio may need custom filtering. It simplifies data collection at the expense of control. If used intelligently (e.g. query "AI" separately to ensure enough AI stories), it could work well. This approach is viable, especially as a quick solution, but one must be mindful of API limits and quality of results.

4. **Approach 4: Techmeme-Style Web Scraping**
   Description: Instead of building our own ranking algorithm fully, leverage an existing tech news aggregator like **Techmeme** (a popular tech news curator site). Techmeme lists the top news and group them by story clusters. The idea would be to scrape Techmeme's daily leaderboards or weekly

archives, and use that as our source of top stories, then summarize those.
Pros: Techmeme essentially does what we want – it finds the hottest tech stories and even shows multiple sources per story. By using it, we inherit their algorithmic/editorial expertise. It could dramatically simplify selection: e.g. pick the top 7 clusters from Techmeme's weekly view (if available). It ensures broad coverage of major news.
Cons: **Scraping complexity and legal/ethical concerns:** Techmeme may not have a public API, so we'd resort to HTML scraping which is brittle and could break if they change their site. It might also be against their terms of service to republish their curated content (though we would only use it to inform our picks, and we'd still link to original articles). Another con is that it removes our own algorithmic learning – we'd be basically reproducing Techmeme's list, which might be fine, but perhaps less original. Also, Techmeme updates constantly; we'd need to be careful to get a representative weekly snapshot. We might also not get enough AI focus if Techmeme doesn't highlight as many AI stories (they usually do, but no guarantee for the half rule).
Probability of Success: High in terms of getting the right stories (Techmeme is quite accurate for top news). However, the approach is somewhat reliant on another site's content structure. It's likely to succeed technically with some effort, but it offloads the core ranking task to an external site, which might not be the intended spirit of the project.

5. **Approach 5: Social Media Trend Analysis**
   Description: Gather signals from social media (Twitter/X trends, Reddit top posts, etc.) for tech topics. For example, track popular tweets from known tech influencers, or see what tech news is trending on Twitter (via hashtags or trending topics API), and also look at subreddits like r/technology or r/MachineLearning for their top posts of the week. Use these signals to choose stories (assuming a story widely discussed on social media is important). Summarize the stories using LLM.
   Pros: Captures the *buzz* in real time. Sometimes social media will highlight important discussions that formal news sites might not (or might lag on). It also gives a measure of public interest/impact (e.g. a news that goes viral on Twitter clearly had big reach). Reddit's upvotes on r/technology is akin to HN's points but for a broader audience.
   Cons: **High complexity and noise:** Social media is noisy; trending topics might be vague (e.g. "#AI" trending doesn't tell which story caused it). Also, API access is problematic (Twitter API is no longer free/cheap, Reddit's API changes, etc.). We'd have to map a trending hashtag or post back to a news story which is not straightforward. There's also a risk of including hype or non-news (memes or controversies that aren't actual "tech developments"). Additionally, implementing this requires a lot of integration (Twitter API, Reddit API) and data parsing, which is overkill for our simple weekly digest.
   Probability of Success: Low to Moderate. While it might catch some important things, the effort to filter signal from noise is high. It complicates the system significantly. It could complement another approach (e.g. as an extra signal in the scoring), but as a primary method it's not reliable enough for a focused "top news" product.

6. **Approach 6: LLM-Driven Selection (Agent)**
   Description: Use an LLM as an "agent" to decide the top stories. For example, feed the LLM a large list of article headlines or brief descriptions from the week (perhaps 50-100 candidates) and prompt it to *"Identify the 7 most important and influential news from this list, especially focusing on AI advancements and major tech developments."* The LLM would then output a list of 7 titles or summaries. Essentially, the LLM does the ranking/selection for us (possibly also summarizing in its answer).
   Pros: This leverages the LLM's knowledge and ability to synthesize. It might pick up on context (e.g. it

"knows" which events are a big deal, especially if it has been trained on a lot of data up to some cutoff). It simplifies our coding – we mainly gather data and then rely on a prompt. It's a very *radical* use of AI in the loop. This could be useful if we trust the LLM's judgment. It can also directly ensure the AI content ratio by the way we prompt ("at least half should be AI-related").
Cons: **Reliability and cost:** If using a powerful LLM like GPT-4 for this reasoning, it's costly and not local. A local LLM might not have up-to-date knowledge or enough "judgment" to accurately know impact (especially since a local model wouldn't have internet access – it can only infer importance from how the headlines sound, which could be misleading). LLMs might also hallucinate – e.g. it might merge two stories or pick something that wasn't actually in the list if the prompt is not carefully constrained. There's a risk that the LLM's output is inconsistent (one run to another). Also, debugging why it chose something might be hard. Essentially, we'd be outsourcing our scoring algorithm to a black-box – which goes against the idea of deterministic selection.
Probability of Success: Moderate. It could work (especially with GPT-4) and yield interesting results, but it's not guaranteed to align with real-world significance all the time. It also undermines the transparency of criteria (we want to be able to justify why a story was chosen – with an LLM agent, it's just what the AI *thinks*). For a production system where consistency and explainability are valued, this approach is risky. It's an innovative alternative but likely not the best for a stable weekly product.

7. **Approach 7:** Hybrid Multi-Source Pipeline (Recommended)
Description: **Combine the strengths of multiple sources and use a custom pipeline (like described in detail above) to algorithmically rank stories, supplemented by LLM summarization. Specifically, use** Hacker News + curated tech news feeds + intelligent scoring**. For instance, gather HN top stories, plus articles from top tech media and possibly an AI-specific feed; deduplicate them into story clusters; use a weighted scoring (taking into account HN votes and multi-source coverage) to select the top 7; then use LLM for summaries. This hybrid approach tries to ensure no major story is missed and balances community interest with media coverage.**
Pros: Comprehensive and balanced. **Hacker News input ensures we capture what developers/ tech insiders find important (often including open-source releases, technical breakthroughs), while the media feeds ensure we don't miss big industry news (product launches, policy, etc.). The custom scoring gives us control to enforce project-specific rules (like AI story quota). It is fully automated and does not heavily rely on any single service (we can swap sources if needed). The architecture is modular and extensible – for example, if in future we want to add weighting for social media mentions, we can incorporate that into the scoring step easily. Additionally, using local LLMs for summarization keeps recurring cost low.**
Cons: **The pipeline is somewhat complex – it involves multiple moving parts (fetching from various APIs/feeds, content parsing, clustering, scoring). This means more code to write and maintain compared to a single-source approach. There's potential duplication of efforts with known aggregators, but we gain customization in return. Testing and fine-tuning will be needed to get the formula for scoring right. Also, initial development time is higher because we integrate several components. However, each component is straightforward (e.g., RSS parsing, or using an existing library for HN API, etc.), so it's more about coordinating them properly.**
Probability of Success: **High. This approach is likely to produce a high-quality weekly list that aligns with user expectations ("top of top" stories, plenty of AI coverage, no fluff). Each part of the hybrid system has been proven individually (e.g., community ranking, multi-source news merging) – we are just combining them. The use of LLM summarization is standard practice in**

similar projects [2] **, so that's a safe bet for quality output. Overall, while requiring more initial effort, this architecture offers the best trade-off between** comprehensiveness, control, and cost-effectiveness**. We consider this the** recommended approach** for LastWeekIn.Tech.

Recommended Solution & Rationale

After evaluating the above options, we recommend **Approach 7: Hybrid Multi-Source Pipeline** as the foundation for LastWeekIn.Tech. This approach is essentially what we detailed in the architecture design sections, and we believe it best fulfills the project requirements:

- **Why Hybrid (HN + News feeds)?:** It ensures broad coverage. Hacker News provides a crowd-curated lens, often highlighting cutting-edge AI releases and developer-focused news, while the official news site feeds ensure we catch corporate and industry news (product launches, acquisitions, policy changes) that might not trend on HN. By merging these, we get a well-rounded set of candidates. Approaches that rely on a single source (like only HN or only one site) would either be too narrow or not sufficiently tuned to tech audiences. The hybrid approach also naturally brings in multiple perspectives on the same story (which improves our confidence that those stories are truly important).

- **Why Custom Scoring vs. Offloading to API/LLM?:** Creating our own scoring algorithm might seem reinventing the wheel, but it grants us *transparency and control*. We can clearly explain why a story was chosen ("It had 250 points on HN and was reported by 3 major outlets") which adds credibility to the product. A NewsAPI or LLM black-box might produce results but without insight or guarantee that they align with our editorial stance (like ensuring enough AI coverage). Our custom logic can enforce those editorial rules explicitly. Moreover, building this logic is a one-time cost and then it runs cheaply on local hardware; relying on external APIs or AI for selection would either incur ongoing costs or possible unpredictability.

- **Use of LLMs:** The recommended solution uses LLMs *strategically* – mainly for summarization (and potentially categorization), which adds value in terms of content quality, but not for the core decision of what's important. This keeps costs low (since summarization can be done with smaller models) and avoids over-reliance on LLM "judgment." At the same time, it fulfills the user's desire to leverage AI capabilities. We will also run these models locally, aligning with the cost constraint. The pipeline essentially uses LLM as a tool within a deterministic framework, which is a sound engineering approach: we get the creativity/linguistic capability of AI where needed, but maintain deterministic logic for selection.

- **Simplicity & Maintainability:** Although hybrid sounds complex, each piece is manageable and uses simple principles (RSS parsing, sorting, etc.). We adhere to KISS by not introducing unnecessary microservices or cloud functions – it's essentially one Python script (or set of scripts) running sequentially. The domain-driven structure makes it easier to extend (e.g., adding a new feed is just adding a new SourceFetcher instance). If something goes wrong or needs tuning (like maybe too many similar stories about a single event are taking multiple slots), we can adjust the algorithm in our code. This is far easier than trying to tune an opaque API or an AI prompt.

- **Reliability:** The static site approach in combination with an offline pipeline is robust. Even if the pipeline fails or we have an issue, the last successful run's content remains on the site (so it won't go

down or show errors to users – worst case, it might not update on a Monday, which can be communicated). The recommended architecture decouples content generation from serving, meaning the user-facing side is extremely stable (just static files on Vercel). This is a proven pattern for low-maintenance websites.

- **Performance considerations:** The hybrid pipeline is efficient enough for a weekly job. HN top stories are at most a few hundred items; RSS feeds similarly. The content extraction and summarization are the heaviest steps, but processing perhaps ~200 articles and summarizing 7 is easily done on a modern laptop within a few hours (likely much less with parallel fetches and a GPU for summarization). So it meets the requirement that it can run on home hardware continuously if needed. Meanwhile, the end-user experience remains fast (just loading one JSON and some text).

In contrast, other approaches either compromise on quality of selection (Approach 1, 3) or introduce complexity without clear benefit (Approach 5, 6). Approach 2 (manual feed list) is actually quite similar to our hybrid but without HN – we include HN because it's too valuable to ignore. Approach 4 (Techmeme scrape) was a close contender, but relying on scraping another aggregator feels less robust and less original in the long run.

Therefore, the recommended architecture is to implement the **Multi-Source Pipeline with Custom Scoring and LLM Summaries** as described. This approach maximizes our probability of success in delivering a useful weekly digest that matches the vision of "only the top of the top" tech stories, especially highlighting AI developments.

Detailed Architecture & Implementation Plan (Hybrid Approach)

Now we outline the detailed spec for implementing the recommended solution, including module breakdowns, data models, and how the pieces interact. This serves as a blueprint for coding (and can be handed to an AI coding assistant or developers to start implementation):

Backend Pipeline Modules

1. **Configuration:** Create a config file or section (could be a Python dict or a YAML) specifying:
2. The list of sources to fetch. For example:

```
sources:
  - type: hackernews
    name: "Hacker News Top"
    limit: 100  # maybe top 100 stories of week
  - type: rss
    name: "TechCrunch"
    url: "https://techcrunch.com/feed/"
  - type: rss
    name: "The Verge"
    url: "https://www.theverge.com/rss/index.xml"
  - type: rss
    name: "Ars Technica"
```

```
    url: "http://feeds.arstechnica.com/arstechnica/index"
  - type: rss
    name: "MIT Tech Review"
    url: "https://www.technologyreview.com/feed/"
  - type: rss
    name: "Wired"
    url: "https://www.wired.com/feed/rss"
  # (Add more as desired)
  - type: newsapi
    name: "NewsAPI Tech"
    query: "technology AND NOT gossip"
    api_key: "... if needed ..."
```

This config-driven approach means we can easily adjust sources without changing code logic – adhering to the Open/Closed principle. Initially, we might hardcode a few sources in code for simplicity, but as a spec it's good to allow configuration.

3. Possibly thresholds or weights for scoring (e.g. define the AI bonus, weights for HN vs sources). But those can also be constants in the code that are easy to tweak.

4. **Data Storage:** Define the SQLite schema if using DB:

```
CREATE TABLE articles (
  id INTEGER PRIMARY KEY,
  source TEXT,
  title TEXT,
  url TEXT UNIQUE,
  published DATETIME,
  hn_points INTEGER,
  hn_comments INTEGER,
  content TEXT,
  is_ai BOOLEAN
);
```

We ensure `url` is UNIQUE so duplicates from different sources don't double insert. Alternatively, we handle uniqueness in code by checking seen URLs. A separate table for stories might not be needed; we can compute stories on the fly. If we want to store final stories (for archiving editions), we could have a `stories` table with fields `week`, `title`, `summary`, etc., but since we output JSON, we might skip that.

5. **Fetching Phase Implementation:**

6. *HackerNewsFetcher:* Use the official HN API via Firebase or the Algolia API. For example, Algolia usage:

```
import requests
url = "http://hn.algolia.com/api/v1/search_by_date?
tags=story&numericFilters=points>50,created_at_i>start_ts,created_at_i<end_ts&hitsPerPage=1(
# This query gets stories in a date range with points > 50.
```

We'd construct `start_ts` as epoch of last Monday and `end_ts` as epoch of this Monday. The response gives JSON with hits including title, URL, points, etc. Parse those into Article objects. If using Firebase API: get top story IDs via `https://hacker-news.firebaseio.com/v0/topstories.json`, then for each ID call `item/{id}.json` and filter by date – but that gives current top, not bounded by week. Algolia is easier for time-bounded search. We might also search by the past week and sort by points descending. Either way, get a list of HN stories with their points. Fill `hn_points` and `hn_comments`.

7. *RSSFetcher:* Use `feedparser` like:

```
import feedparser
feed = feedparser.parse(feed_url)
for entry in feed.entries:
    title = entry.title
    url = entry.link
    published = entry.published_parsed or entry.updated_parsed
```

Create Article for each. Note: Some RSS feeds include full content in `entry.content[0].value` (HTML) or summary. We could store that in content temporarily if provided, but likely we'll still fetch the article to be safe.

8. *NewsAPIFetcher:* Use the `newsapi` Python client or direct HTTP. For example:

```
GET https://newsapi.org/v2/everything?
q=technology&language=en&from={last_week_date}&to={today_date}
&sortBy=popularity&pageSize=100&apiKey=XYZ
```

That could return many articles. We filter for English tech news. Or use `top-headlines?category=technology&country=us` for a broad snapshot. But everything with sort by popularity might be better for a week scale. Parse results into Article objects.

9. All fetchers should handle errors (e.g., network issues) gracefully – maybe retry once, otherwise log and continue with others. The pipeline shouldn't abort just because one feed failed; we can proceed with what we have and still produce output, albeit maybe missing one source. Logging is important for debugging (we'll log how many articles fetched from each source, etc.).

10. After fetching, we insert these Articles into SQLite (if using it) or at least combine into one list. Use the DB to enforce uniqueness on URL (catch constraint exceptions or do SELECT to skip existing).

11. **Content Extraction Phase:**
    Iterate over articles without content (or all, if we didn't get content from feeds). For each:

12. Use newspaper3k:

```
from newspaper import Article as NewsArticle
art = NewsArticle(article.url)
try:
    art.download()
    art.parse()
except Exception as e:
    log(f"Failed to download {article.url}: {e}")
    continue
content = art.text  # the extracted text
article.content = content
```

We also get `art.authors`, `art.publish_date` if needed. If `content` ends up very short (like < 50 words), maybe skip or mark article as possibly useless (some feeds might have short announcements).

13. Rate limiting: possibly introduce a short sleep (like 1 second) between fetches to not overload servers, or use `asyncio` with a semaphore to parallelize but within a limit. Because we might fetch ~100 articles, doing it sequentially could take a couple minutes, which is fine.

14. Update the DB with the content (e.g. an UPDATE statement for that article's row).

15. **Determining AI vs General Category:**

16. We can define a simple function `is_ai_related(article)` that checks the title and maybe first 100 words of content for AI keywords: e.g. regex search for `r"\b(AI|Artificial Intelligence|machine learning|neural network|large language model|OpenAI|Google AI|GPT|LLM)\b"` (case-insensitive). If found, mark `article.is_ai = True`.
17. Alternatively, use a pre-trained zero-shot classifier (like BART-MNLI) with labels ["AI", "Not AI"] by feeding the article's title or summary. But this is probably overkill when keywords suffice.
18. If using an LLM, we could do one pass after summarization to classify each summary as AI or not (since summary is short and to the point). But to enforce the half rule during selection, we need this info earlier. So do it based on title/content.

19. This sets a flag for each Article which can later be applied to Story (if any article in a story is AI-related, likely the story is AI-related – especially if a story has mixed, we err on marking AI if there's any substantial AI mention).

20. **Story Clustering Phase:**

21. We'll gather the articles into clusters. Pseudocode:

```
stories = []
seen = set()  # set of article ids or URLs already clustered
for article in articles_sorted_by_date:  # or maybe by source importance
    if article.id in seen:
        continue
```

```
    # Start a new story cluster
    story_articles = [article]
    seen.add(article.id)
    for other in articles:
        if other.id not in seen:
            if titles_similar(article.title, other.title):
                story_articles.append(other)
                seen.add(other.id)
    # Create Story object
    story = Story()
    story.articles = story_articles
    # Determine story.title (maybe the title of the first article or a
combination)
    story.title = choose_best_title(story_articles)
    # Determine story.is_ai (True if any article.is_ai True)
    story.category = "AI" if any(a.is_ai for a in story_articles) else
"General"
    # Compute HN points aggregated
    story.hn_points = max(a.hn_points or 0 for a in story_articles)
    story.num_sources = len({a.source for a in story_articles})
    story.published = min(a.published for a in story_articles)  # earliest
publish (or latest? could store both)
    stories.append(story)
```

22. The `titles_similar` function could use a simple heuristic: one approach is to strip punctuation and stopwords then check if one title contains a significant subset of the other's words. For example, if title A has length 5 words after filtering, and at least 3 of those words appear in title B, they might be the same story (especially if those 3 include a proper noun or unique term). We can also look for common substrings (like anything beyond 15 characters in common). This might join some things that are a bit loosely related, but we can refine if needed.
23. Alternatively, we could use a library like `sklearn.feature_extraction.text TfidfVectorizer` on all titles and then do cosine similarity. If >0.3 similarity, group them. But that might be heavy for just a few hundred strings.
24. `choose_best_title` could simply pick the longest title (assuming it might be most descriptive), or the title from the article with the highest HN points (meaning that phrasing got the most attention, but on HN often the title is the same as original). We could also prefer titles from certain sources (e.g. if one is from a more formal news site vs a forum title). Probably using the first article's title is fine if our article list is sorted by something like source priority or HN points.

25. Each Story now represents one news event.

26. **Scoring Phase:**

27. Implement the scoring formula through a function or within StoryScorer class. For each Story:

```
score = 0
score += story.hn_points_score()  # convert HN points to score portion
```

```
score += story.num_sources * 5
if story.category == 'AI':
    score += 3
# Optionally consider recency:
days_ago = (today_date - story.published).days
if days_ago < 3:
    score += 1  # a small boost for very recent stories, to not
disadvantage them
story.score = score
```

28. The HN points to score conversion could be, as earlier, something like `min(hn_points, 300) / 10` (so max 30 points from HN factor). Or use log: `score += math.log2(hn_points+1) * 5` for diminishing returns. We'll likely experiment. We could also incorporate comments count similarly if desired (comments often reflect interest too).

29. After this, we sort stories by `score` descending.

30. **Selection Phase:**

31. Take the first 7 of the sorted list as `top_stories`. If less than 7 stories exist (unlikely given many inputs), then we take all available (though then the concept fails – but realistically there will always be at least 7 news items in a week).
32. Apply the AI quota check:

```
ai_count = sum(1 for s in top_stories if s.category == 'AI')
if ai_count < 3:
    # number to replace = 3 - ai_count (if we want at least 3)
    needed = 3 - ai_count
    # Find the top 'needed' stories beyond the top 7 that are AI
    candidates = [s for s in stories[7:] if s.category == 'AI']
    if candidates:
        candidates.sort(key=lambda s: s.score, reverse=True)
        for i in range(min(needed, len(candidates))):
            # find the lowest-ranked non-AI story in current top_stories
            replace_index = max(idx for idx, s in enumerate(top_stories) if
s.category != 'AI')
            if replace_index is not None and candidates[i].score >
top_stories[replace_index].score * 0.5:
                # Only replace if candidate is at least reasonably scored
(e.g. >50% of the story it's replacing)
                top_stories[replace_index] = candidates[i]
            else:
                break

# After replacement, we might reorder top_stories by score again if order
matters.
```

33. We included a condition to avoid replacing a much higher scored story with a low one just for AI – ensuring some quality threshold. This logic can be fine-tuned.

34. The result is our final list of 7 Story objects.

35. **Summarization Phase:**

36. For each Story in top_stories:
    - Determine the text to summarize. Perhaps use the content of the first Article in `story.articles` (assuming that's the representative main source). We should ensure we use a content that is sufficiently detailed. If multiple articles are in the story, we might choose the longest content one.
    - Pass the text to the summarizer. Possibly split the text if it's longer than model max tokens (e.g., for BART, maybe feed at most ~1024 tokens).
    - Get the summary text. If using the pipeline:

      ```
      summary = summarizer(article_text, max_length=120, min_length=30,
      do_sample=False)
      summary_text = summary[0]['summary_text']
      ```

      Where `max_length` 120 tokens ~ roughly 3-4 sentences, we can tweak.
    - If using a custom model interface, generate accordingly.
    - Assign `story.summary = summary_text`.
    - Also record `story.source = article.source` or maybe if multiple, we might list one primary source. For UI, it might be nice to show which outlet this story is from. If multiple sources, maybe choose one (like the one whose title we used).

37. After all, we have top_stories each with title, summary, category, source, and link. For the link, maybe also choose one representative URL (preferably a *canonical* news source rather than HN link – e.g. if HackerNews had a high rank but the story is actually a NYTimes article, use the NYTimes URL. If the story is a discussion around a GitHub repo and only HN is the source, we might link to the HN discussion or the repo itself depending on context. Usually, HN's URL is an external link, which we have, so link to the external content rather than the HN page).

38. **Output JSON and Deployment:**

    - Format the data as JSON (as shown in example earlier). We'll include:
    - `week` – we can use the date of the Monday for that week, or an ISO week number, but date is simplest. If the pipeline runs on Monday, that date is the new edition date.
    - `stories` – an array of 7 items, each with fields: `title`, `summary`, `url`, `source`, `category` (maybe), and possibly `rank` or `score` if we want to keep it (not needed for display but could be useful for debugging or future use).
    - Write this JSON to a file in the project (e.g. `data/2025-11-03.json`). Also (or alternatively) update a `latest.json` symlink or copy for easier fetch by frontend.
    - If maintaining an archive, append the filename to an archive list. In the frontend, we could then allow selecting past weeks. Initially, perhaps not needed.
    - Use Git operations to commit this file: e.g.

```
git add data/2025-11-03.json
git commit -m "Add edition 2025-11-03"
git push
```

If the pipeline itself is run manually by the developer, they can handle this. If we wanted full automation, we might incorporate a GitHub Actions workflow that triggers weekly to run the pipeline on a runner. But since the user mentioned running on local laptops, we assume manual or local scheduling and then pushing.

- Once pushed, Vercel triggers a deployment. The static site will then serve the new content.

39. **Logging & Monitoring:** (Not explicitly asked for, but good practice)

    - The pipeline should log key events to console or a log file: e.g., "Fetched 120 articles (HN:30, TechCrunch:20, ...)", "Clustered into 80 stories", "Top 7 preliminary: Titles X, Y, Z... (4 AI stories)", "Replaced 1 story to meet AI quota", "Summaries generated", etc. This will help in debugging and verifying the pipeline is working as intended each week.
    - If an error occurs in one part (say one site fails or summarizer times out), the pipeline should catch exceptions and continue where possible, so that one issue doesn't prevent producing any output. Perhaps have a fallback summary if LLM fails (even if just a truncated content). The goal is robust automation – no manual intervention ideally.

Frontend Implementation Details

For the Vue.js frontend, we provide a plan focusing on simplicity, as it's mainly static content display:

1. **Project Setup:** Use Vue 3 with a bundler (Vite is a good choice for speed). Initialize a project with `npm init vue@latest` or similar. We might not need heavy state management, so avoid Vuex/pinia unless necessary. Install Vuetify if using it: `vue add vuetify` or follow Vuetify setup for Vue 3. Also include the Titillium Web font in `index.html` via a `<link>` to Google Fonts or in CSS `@import`.

2. **Global Style:** Set up some basic global styles (like applying Titillium Web to body, maybe setting a light background color, etc.). If using Vuetify, configure its theme to use a certain primary color that fits a tech aesthetic (blue or teal often works) and set the font family.

3. **Components:**

4. **StoryCard.vue:** A single-story component. Template might look like:

```
<v-card class="ma-3 pa-3" outlined>
  <v-card-title class="headline">{{ story.title }}</v-card-title>
  <v-card-subtitle class="mb-2">
    <span>{{ story.source }}</span>
    <span v-if="story.category === 'AI'" class="category-chip">AI</span>
    <span class="date">{{ formattedDate }}</span>
  </v-card-subtitle>
```

```
  <v-card-text>{{ story.summary }}</v-card-text>
  <v-card-actions>
    <v-btn small text color="primary" :href="story.url" target="_blank">
      Read Original <v-icon small>mdi-open-in-new</v-icon>
    </v-btn>
  </v-card-actions>
</v-card>
```

(If not using Vuetify, we'd use `<div>`s and style accordingly. The concept remains: title, source/category info, summary, link button.)
The component receives a `story` prop. It might compute `formattedDate` if we include publication date (if provided in JSON, we could). Or if not, we might omit date to avoid clutter. The category chip ("AI") can be a styled span or a Vuetify `<v-chip>`.

5. **StoryList.vue:** This parent component fetches the stories and iterates StoryCard. Template:

```
<div>
  <StoryCard v-for="story in stories" :key="story.title" :story="story" />
</div>
```

In the script, it might fetch the JSON in `mounted()` hook:

```
mounted() {
  fetch('/data/latest.json')
    .then(res => res.json())
    .then(data => {
      this.stories = data.stories;
      this.week = data.week;
    })
    .catch(err => { console.error("Failed to load stories", err); });
}
```

Also store `week` (for display if needed).

6. **App.vue (Main page):** Use a simple layout: maybe a toolbar at top and the StoryList. For example, using Vuetify:

```
<v-app>
  <v-app-bar app color="primary" dark>
    <v-toolbar-title>LastWeekIn.Tech</v-toolbar-title>
    <span class="flex-grow-1"></span>
    <span v-if="currentWeek">Week of {{ currentWeek }}</span>
  </v-app-bar>
  <v-main>
    <v-container>
      <StoryList @loadedWeek="currentWeek = $event" />
```

```
    </v-container>
  </v-main>
  <v-footer app dense class="text-center grey lighten-4">
    <span>&copy; 2025 LastWeekIn.Tech – Updated every Monday</span>
  </v-footer>
</v-app>
```

If not using Vuetify, a simpler structure with plain HTML: a header with site title, maybe a subheader with the week, then the list of stories, and a footer. Material Design can still be approximated with CSS (e.g. using CSS Grid/Flex for card layout, shadows for cards).
App.vue will receive from StoryList the current week (we can emit when data loaded). Or App itself fetches and passes down. Either way works. Possibly simpler: App.vue does the fetch and passes stories as props to StoryList. But either is fine.

7. **Routing (Optional):** If we plan to allow viewing past weeks, we might incorporate Vue Router. For example, route `/2025-11-03` loads that week's JSON. In that case, StoryList would fetch based on route param. But since initial spec doesn't require historical browsing, we can skip routing for now – or implement a very basic one if we want to future-proof. Without routing, the site essentially only shows the latest week (maybe with a note "Week of X" to indicate date).

8. **Testing UI:** We should test with dummy data to ensure the layout looks good for 7 items. Also test on mobile viewport – cards likely stack which is fine. The Titillium font might need a fallback (like sans-serif) if not loaded for some reason. We will also ensure the external link button works and has `target="_blank"` to not navigate away from our site.

9. **Accessibility & SEO:** Because it's mostly text content, it's inherently accessible (screen readers can read the summaries and links). We should add proper HTML semantics (e.g., use `<main>` for the container of stories, use `<article>` tags for story card maybe, and the link should have an `aria-label` like "Read full story on [Source]"). SEO wise, if it's a SPA without pre-render, search engine bots might not see content. This is a downside of SPA. If this is a concern, we could generate a static index.html with the content embedded each week (server-side rendering). But as mentioned, that adds complexity. Alternatively, use a service like prerender.io if needed. Given time and simplicity, we may accept that SEO is secondary (people will likely find the site via word of mouth, not search, at least initially).

10. **Material Design nuances:** We should use consistent padding, margins. Vuetify or other component libs handle a lot of that. If doing custom, set e.g. `.card { border-radius: 8px; box-shadow: 1px 1px 5px #ccc; margin: 16px 0; padding: 16px; }` etc. Use the theme's primary color for link or accent. The category chip for "AI" could have a distinct color (maybe a light blue or green background with white text, small rounded pill shape) to subtly highlight AI stories. This will visually communicate the AI focus to users.

11. **Integration with Data:** Ensure the path to JSON is correct. If we deploy to Vercel with the JSON in `public/data/latest.json`, it will be served at `/data/latest.json`. Or we might embed it by importing if we treat it as a module (but simpler to fetch as shown). Using the event of commit, the JSON will be updated. No caching issues should arise if we maybe append a query param like `?`

`v=20251103` if needed, or set cache headers. Vercel by default may serve static with caching – perhaps we configure a short max-age or rely on file content hash. If using the same filename always (latest.json), we might need to ensure browsers don't cache it across weeks. We can set meta no-cache or version it (one trick is to have the frontend request `latest.json?week=2025-11-03` dynamically, but since it's static hosting, better to just use a unique filename per edition and maybe symlink latest). Simpler: incorporate the week in the filename and have the frontend URL updated in code when we push (like we update a constant in code to point to that file). But that requires rebuilding the app each time as well – which we do via commit anyway. So actually, we could skip fetching altogether and just bake the data as a constant in the app at build time. However, fetching decouples data from build, which is cleaner. We might just ensure to disable caching for that JSON via a header if possible. We'll mention that as a deployment consideration.

Example Scenario (End-to-End):

To illustrate how everything works together, let's walk through an example "week update":

- **Sunday, Nov 30, 2025 night:** The pipeline is scheduled to run (maybe via cron) on the developer's laptop. It starts by calculating the date range (Nov 24-30) and fetching sources. It pulls, say, 80 HN stories (some filter of points>20), 50 from TechCrunch, 40 from The Verge, etc., totaling ~200 raw articles. It inserts them into the DB. Duplicate URLs are dropped (maybe a TechCrunch article that was popular on HN appears in both sets – the DB unique constraint ensures we have it once).
- It then downloads each article's content. It runs newspaper3k; for some sites that might fail due to Cloudflare, the pipeline logs and maybe tries an alternative (but let's assume most succeed). Now the DB has content for ~180 articles (some might be skipped if unreachable).
- Clustering groups these into, say, 150 stories (since many are unique, but a few big ones cluster e.g. "New AI Chip X" had 3 sources writing about it).
- The scoring assigns high points to a story "OpenAI's New Model" because it had 250 HN points and 4 sources. Another story "Apple's AR Glass Launch" had 5 sources but wasn't on HN (score from sources count). It sorts them; the top 10 look plausible. It finds that in the top 7, there are 2 AI stories (OpenAI model and maybe one about a research breakthrough). It needs at least 4, so it looks at positions 8-12 and finds two more AI stories (maybe not as high overall, but decent). It swaps them in for the bottom of the seven which were non-AI (perhaps dropping a less critical non-AI story). Now we have 4 AI, 3 other tech in the final list.
- Summaries are generated: for each of the 7, the pipeline feeds the content to the summarizer model. It produces summaries which the pipeline trims to ensure brevity. e.g., for the OpenAI story it outputs a neat 2-sentence summary. The pipeline possibly post-edits trivial stuff (like removing "Read more on …" if the model had any).
- JSON is created with those 7 items. The pipeline writes `2025-11-30.json` and also updates a `latest.json` to copy that (or a config file with current week). It then commits these files to git and pushes.
- **Monday, Dec 1, 2025 early morning:** Vercel picks up the commit, builds the app, and deploys.
- A user visiting LastWeekIn.Tech on Monday sees the new content: The page header says "Week of Nov 30, 2025" and below are 7 cards. The first card maybe has title "OpenAI Releases GPT-6, Paving Way for Advanced Reasoning" with a tag "AI" and summary "OpenAI unveiled its GPT-6 model, significantly outperforming GPT-4 in benchmark tests. Experts say this could enable more complex AI applications and sparks a new wave of AI-powered tools." Then a "Read Original" link (maybe to an article on TechCrunch or OpenAI's blog).

- They scroll and see other stories (some AI like “New AI chip from NVIDIA doubles training speed”, some general tech like “Apple launches AR Glasses” etc.), each with a crisp summary.
- If it’s well-designed, the user can quickly scan these summaries in a minute or two and feel caught up for the week, and click any that they want full details on.
- During the week, nothing changes on the site until next Monday when the process repeats.

This scenario highlights how the architecture meets the need: minimal manual work (just monitor the pipeline), static serving, and curated quality content.

Conclusion

We have specified a comprehensive plan for **LastWeekIn.Tech**. The solution involves a robust backend pipeline to combat information overload by intelligently selecting and summarizing the week's top tech (especially AI) news, and a sleek frontend to present it. The design follows best practices (DDD boundaries, modularity, DRY code reuse) while keeping things as simple as possible (KISS) for both implementation and maintenance.

By comparing multiple approaches, we concluded that a hybrid curation strategy gives the best outcome, balancing coverage and feasibility. The architecture is detailed enough to guide implementation (with specific classes, functions, and even pseudocode laid out), and flexible enough to evolve (new sources, different LLMs, or expanded features like archives can be added with minimal refactoring thanks to the clean separations).

In summary, LastWeekIn.Tech will be built on a **Python (data pipeline) + Vue.js (frontend)** stack, using **local LLMs for NLP**, and deployed as a **static site** on Vercel. Once implemented, users will have a one-stop weekly briefing of tech’s most impactful stories, presented in a concise and modern format – solving the problem of information overload in an elegant, automated way.

---

1 4 5 7 How I Built a Production-Ready LLM News Aggregator from Scratch | by Lijo Raju | Medium
https://medium.com/@lijoraju/how-i-built-a-production-ready-llm-news-aggregator-from-scratch-015ebd08d566

2 3 6 8 Tech News Aggregator: The Verge & TechCrunch RSS to Notion with GPT-4 Summaries | n8n workflow template
https://n8n.io/workflows/8600-tech-news-aggregator-the-verge-and-techcrunch-rss-to-notion-with-gpt-4-summaries/